MACHINE LEARNING OPERATIONS | ASSIGNMENT 01 | OCTOBER 2026

Heart Disease Classification

An end-to-end MLOps workflow for the UCI Cleveland dataset

Course	Dataset	Records	Target
MLOps AIMLCZG523	UCI Heart Disease	303	Diagnosis > 0

Executive summary

This project builds a repeatable path from the supplied clinical data to a model-backed API. The source diagnosis is recoded as absence (0) versus presence (1). One sklearn pipeline owns missing-value handling, feature scaling, categorical encoding, and classification, so training and inference share the same transformations. Logistic Regression and Random Forest are tuned with stratified cross-validation; mean cross-validation ROC-AUC selects the model. The trained artifact is served by FastAPI, while MLflow records experiments and Prometheus metrics feed a provisioned Grafana dashboard.

This report records measured model results and local runtime checks. Docker Compose served health and prediction requests, Prometheus reported the API target as UP, and Grafana received metrics. Docker Desktop Kind ran two API replicas, with health, prediction, and metrics verified through kubectl port-forward. No public cloud endpoint is used.

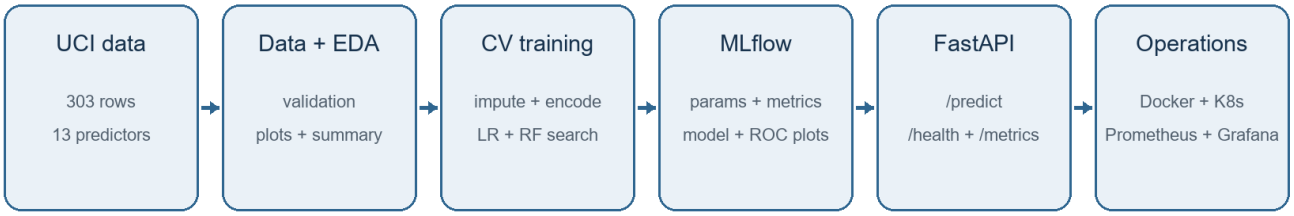


Figure 1. Data, training, serving, and monitoring architecture.

01 | DATA ACQUISITION

Dataset and data contract

The project uses the processed Cleveland subset in the UCI Heart Disease archive. The legacy file has 14 comma-separated fields without a header: 13 predictors and one diagnosis value. The supplied archive is retained at the workspace root; data/raw/processed.cleveland.data is the focused input used by the scripts. The download helper fetches the UCI ZIP and extracts only this named member, printing SHA-256 hashes for provenance.

Acquisition was run from the UCI endpoint. The downloaded archive SHA-256 matched the supplied archive (b17cd273da9ce1caa4710fce80227ea454d4dbf9fcbc8e6a9121672751563adc); the extracted Cleveland file hash is a74b7efa387bc9d108d7d0115d831fe9b414b29ae7124f331b622b4efa0427c8.

Data check	Observed	Handling
Records	303	Keep usable diagnosis rows
Predictors	13 clinical fields	Preserve source order and names
Unknown values	6	Keep missing until model fit
Diagnosis	Ordinal values 0-4	Binary label: 0 vs 1-4
Duplicates	0	Remove exact duplicate rows

Target definition

Diagnosis 0 is absence, while values 1, 2, 3, or 4 represent presence. This explicit transformation is implemented in src/data.py and tested. Predictors are age, sex, chest-pain type, resting blood pressure, serum cholesterol, fasting blood sugar, resting ECG, maximum heart rate, exercise angina, ST depression, slope, major vessels, and thalassemia category.

Six feature values are missing in the source (ca: 4, thal: 2). These cells remain null through data splitting. Imputers are fitted inside each training fold so validation and holdout statistics do not leak into preprocessing.

02 | EXPLORATORY ANALYSIS

Feature distributions and class balance

The first EDA view summarizes all 13 predictors after target mapping. Continuous measurements show patient variability, while coded fields such as chest-pain type and thalassemia type are discrete categories even though they are stored as numbers. This distinction informs the model pipeline: measurements are scaled; clinical codes are one-hot encoded.



Figure 2. Histograms for Cleveland clinical features.

Target	Meaning	Patients	Share
0	No diagnosis	164	54.1%
1	Diagnosis present	139	45.9%

The target is moderately imbalanced. Accuracy alone would be incomplete, so precision, recall, and ROC-AUC are reported. Both candidate classifiers use class weighting.

02 | EXPLORATORY ANALYSIS

Associations and source missingness

The correlation heatmap is descriptive: it shows linear associations among numeric encodings and the binary target. Ordinal category codes can create correlations that do not represent true distances between categories, so the figure orients review rather than driving feature selection.

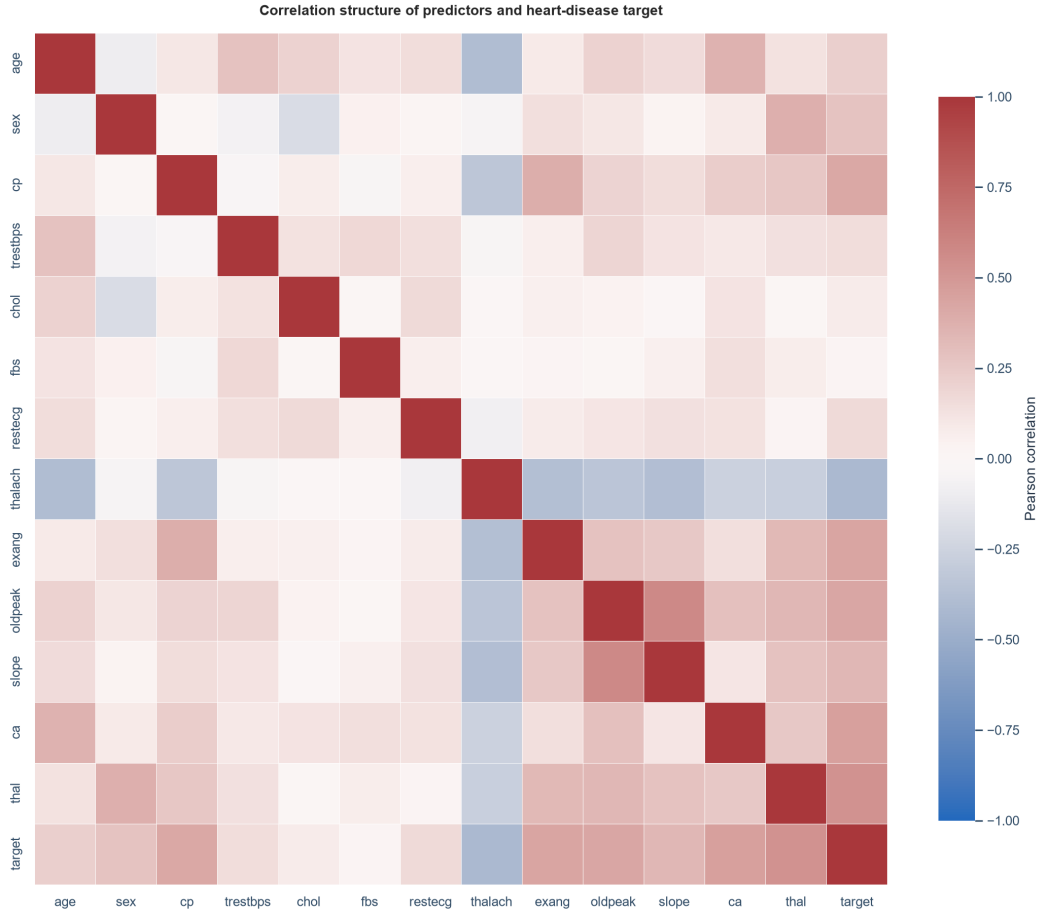


Figure 3. Pearson correlation matrix, including the binary target.

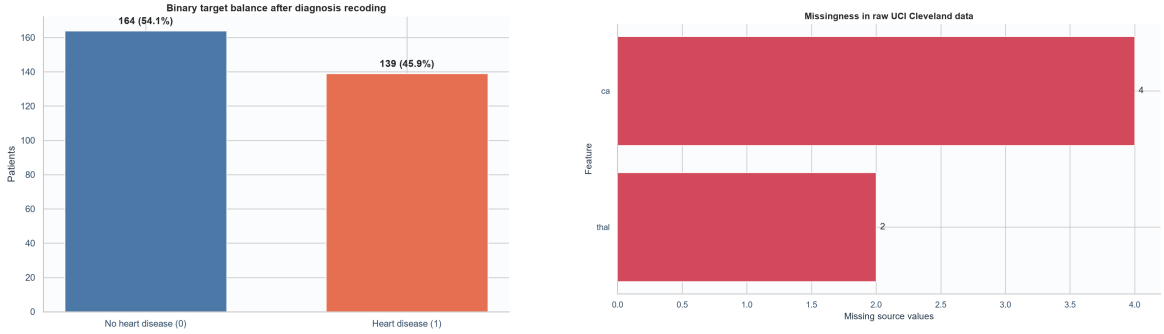


Figure 4. Class balance (left) and raw missingness (right).

The missingness chart uses the raw source, so model-time imputation cannot conceal data quality. Plots and summary counts are regenerated by scripts/eda.py.

03 | FEATURE ENGINEERING

Leakage-safe preprocessing and model selection

A ColumnTransformer routes five measurements (age, resting blood pressure, serum cholesterol, maximum heart rate, and ST depression) through median imputation and StandardScaler. Eight coded fields pass through most-frequent imputation and one-hot encoding. Unknown categories at inference are ignored safely. All transformations are inside the classifier pipeline, so each cross-validation fold learns its own imputation and scaling statistics.

Candidate	Role	Search dimensions
Logistic Regression	Interpretable linear baseline	C; balanced class weights
Random Forest	Nonlinear tree ensemble	Tree count, depth, leaf size

Evaluation protocol

A stratified 80/20 split uses a fixed random seed. GridSearchCV tunes each candidate on the training portion with five stratified folds (three in the CI smoke run). ROC-AUC is the refit metric. Accuracy, precision, recall, and ROC-AUC means and standard deviations are retained. The untouched holdout reports all four metrics once. The selected pipeline is refit on all 303 rows for API use.

The holdout is 61 records and the training portion is 242. These estimates come from a small historical sample and are not a stable clinical performance guarantee.

04 | MODEL RESULTS

Cross-validation and holdout performance

Table 1. Training-set cross-validation scores: mean +/- standard deviation.

Model	Accuracy	Precision	Recall	ROC-AUC
Logistic Regression	0.835 +/- 0.043	0.831 +/- 0.016	0.802 +/- 0.104	0.899 +/- 0.033
Random Forest	0.814 +/- 0.050	0.805 +/- 0.046	0.785 +/- 0.085	0.898 +/- 0.046

Table 2. Held-out 20% test results.

Model	Accuracy	Precision	Recall	ROC-AUC
Logistic Regression	0.918	0.960	0.857	0.974
Random Forest	0.902	0.958	0.821	0.965

Selection

Logistic Regression is selected because it achieved the highest mean cross-validation ROC-AUC (0.899). The margin over the other candidate is 0.0004, small relative to fold variation, so there is no decisive separation. The ranking rule was fixed before holdout comparison; the holdout did not select hyperparameters.

These are experimental estimates on a small historical sample. They are not medical thresholds and should not guide patient care.

05 | EXPERIMENT TRACKING

MLflow and reproducible artifacts

Each candidate run records the model name, selected hyperparameters, cross-validation metrics, holdout metrics, fitted pipeline, and holdout ROC curve. The default tracking URI is a local file store under mlruns/. Run mlflow ui from the repository root to inspect the experiment. metrics.json and model_comparison.csv make results readable without opening the UI. Model signatures use numeric fields that can represent missing values.

Artifact	Contents	Use
heart_disease_model.joblib	Preprocessor and selected estimator	API inference
model_metadata.json	Family, parameters, features, time	Model identity
metrics.json	Data snapshot, selection, CV and holdout	Reporting and audit
model_comparison.csv	One row per candidate	Quick comparison
*_roc.png	Holdout ROC curve per model	Visual assessment
MLflow runs	Parameters, metrics, model and plots	Experiment history

Reproduction controls

The shared schema parser keeps acquisition, EDA, and training aligned. Fixed random seeds control splitting, folds, and estimators. The serialized pipeline preserves fitted preprocessing. requirements.txt and the CI workflow define the environment. The tracking store is local; a shared server and object store would be needed for team-wide tracking.

06 | MODEL SERVING

FastAPI request and response contract

The API loads the packaged sklearn pipeline from MODEL_PATH and accepts one JSON object with the 13 named predictors. Pydantic validates required fields and plausible ranges. ca and thal may be null; the trained pipeline imputes them. The response returns a 0/1 prediction, a class label, and the estimated positive-class probability. This probability has not been calibrated or validated for clinical use.

Example request with two optional values omitted:

```
{
"age": 63, "sex": 1, "cp": 1, "trestbps": 145,
"chol": 233, "fbs": 1, "restecg": 2, "thalach": 150,
"exang": 0, "oldpeak": 2.3, "slope": 3, "ca": null, "thal": null
}
```

POST /predict | GET /health | GET /metrics

Endpoint	Purpose	Expected behavior
/predict	One prediction	Prediction, label, confidence, version
/health	Readiness	503 when model artifact is unavailable
/metrics	Monitoring	Prometheus request, latency, class counters

Privacy and logging

Application logs capture method, route, status, and elapsed time, but omit request bodies. The local Python process and Docker Compose smoke checks returned /health status ok and valid /predict responses for samples/predict.json. The Docker container also returned HTTP 200 from /metrics. Kubernetes health, prediction, and metrics endpoints were verified separately through kubectl port-forward.

07 | CI AND QUALITY

Automated checks and local verification

GitHub Actions installs the requirements file, runs Ruff, executes pytest, trains both models in a smaller CI search mode, and uploads the model, metrics, plots, and MLflow store. A lint, test, or training failure fails the workflow.

Check	Coverage	Observed status
Ruff	Imports and common correctness checks	Passed (ruff check src app scripts tests)
Pytest	Data, recoding, pipeline, API contract	Passed (7 tests)
Full training	Five-fold search, metrics and model	Passed (full 5-fold comparison; Logistic Regression selected)
API smoke	Health and sample prediction	Passed (uvicorn /health and sample /predict)
Remote workflow	Workflow artifact retained per run	Passed (recorded GitHub Actions run #4 on commit e808c1d; workflow concluded successfully)

Reproduction commands

```
ruff check src app scripts tests

python -m pytest

python scripts/train.py --fast
```

Current GitHub Actions status: Passed (recorded GitHub Actions run #4 on commit e808c1d; workflow concluded successfully). The recorded passing workflow run is documented in docs/screenshots/github-actions-ci-passed.png.

08 | DEPLOYMENT AND OPERATIONS

Container, Kubernetes, and monitoring

The Docker image packages the API and serialized model. Docker Compose starts the API with Prometheus and Grafana. The API exports request totals, latency histograms, and prediction counts. A provisioned dashboard shows route request rate, p95 latency, prediction counts, and 5xx ratio. Kubernetes manifests specify two replicas, health probes, resource bounds, scrape annotations, and a LoadBalancer Service.

Evidence item	Prepared configuration	Remaining runtime proof
API container	Dockerfile; health, prediction, and metrics requests	Passed locally
Observability	Prometheus target UP; Grafana dashboard	Passed locally
Kubernetes	Two ready replicas; endpoints via port-forward	Passed locally
Public deployment	No cloud-specific resource is assumed	Not performed; no public URL
Video and screenshots	Four evidence screenshots in docs/screenshots	Short demo video remains

Conclusion and limitations

Docker Compose and the local Docker Desktop Kind cluster have both been exercised. The Kubernetes Deployment reached two ready replicas, and /health, /predict, and /metrics were verified through port-forwarding. Prometheus, Grafana, Kubernetes, and GitHub Actions screenshots are included in the repository. The current GitHub Actions result is Passed (recorded GitHub Actions run #4 on commit e808c1d; workflow concluded successfully). No public cloud endpoint is claimed; a short demo video and the course submission remain for the student. The dataset has 303 historical records and is not suitable for diagnosis or health decisions.

References

UCI Machine Learning Repository. Heart Disease dataset, DOI 10.24432/C52P4X. archive.ics.uci.edu/dataset/45/heart+disease

MLflow. Tracking API documentation. mlflow.org/docs/latest/tracking/tracking-api

scikit-learn. ColumnTransformer documentation. scikit-learn.org/stable/modules/generated/sklearn.compose.ColumnTransformer.html